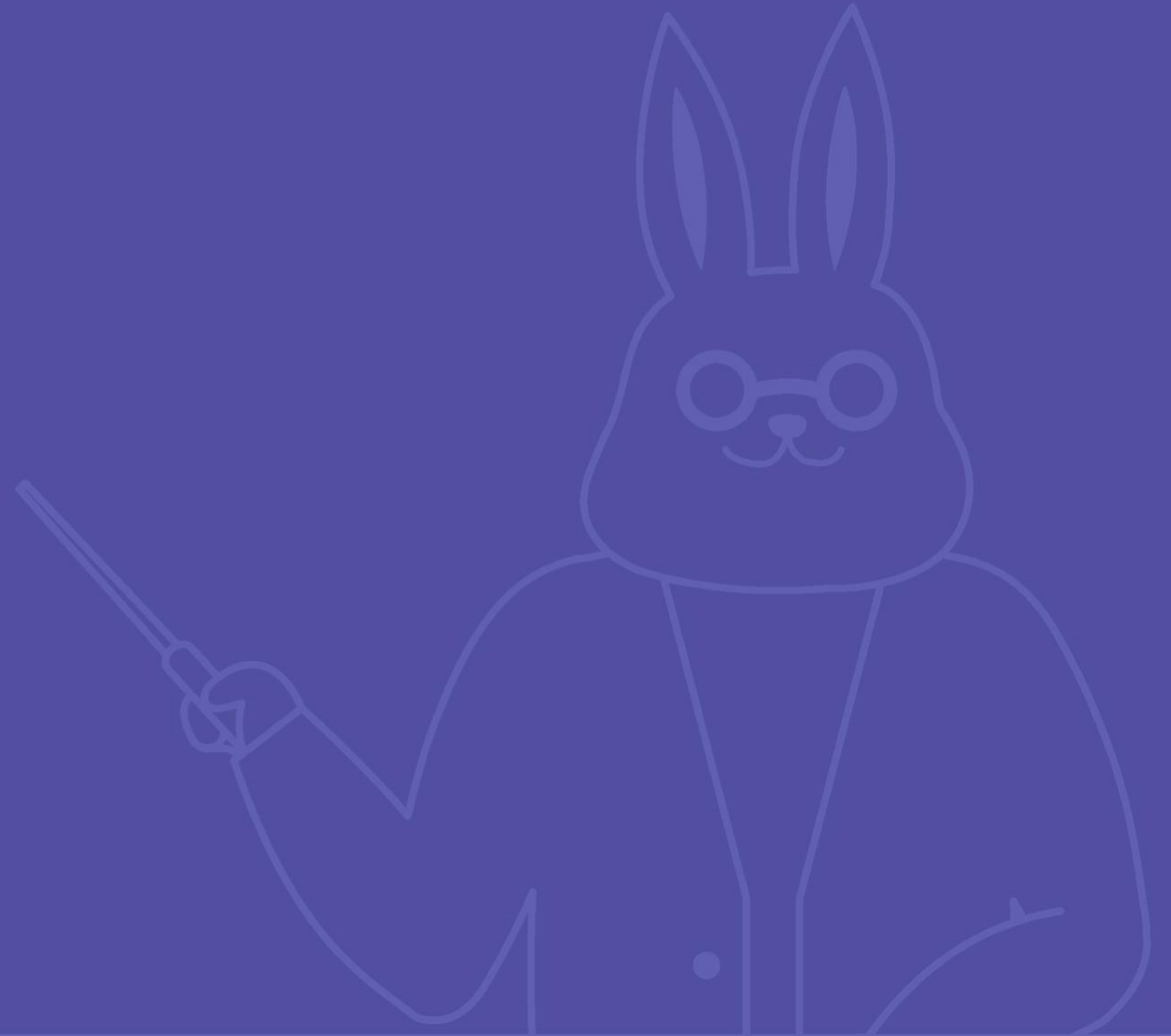




Flask 기초

06 Flask 프로젝트 1 – 익명게시판 만들기



목차

- 01. 프로젝트 구상하기 - 익명 게시판
- 02. 데이터베이스 구성하기
- 03. 화면 만들기
- 04. 기능 만들기 1 - 글 쓰기
- 05. 기능 만들기 2 - 글 읽기
- 06. 기능 만들기 3 - 좋아요 누르기

수강목표

프로젝트를 기획 할 수 있다.

하나의 프로젝트를 설정하고, 분석하고 만들 수 있다.

프론트엔드와 백엔드 간의 통신을 만들 수 있다.

프론트엔드와 백엔드를 구분하고, 각각 요청/응답을 처리할 수 있다.

Flask에서 ORM을 활용해 데이터를 다룰 수 있다.

Flask-SQLAlchemy를 사용해서 ORM 모델을 다루고 데이터를 저장/업데이트/읽기를 할 수 있다.

01

프로젝트 구상하기 - 익명 게시판



✓ 프로젝트 구상하기

개발을 할 때 생각해야 될 것 중 하나는
'**무엇을 어떻게 만들 것인가?**' 입니다.

잘 짜인 구상은 프로젝트를 진행하는 데 많은 도움이 됩니다.

✓ 어떤 프로젝트를 만들까?

주제 : 모두가 이용 할 수 있는 익명게시판

우리가 만들 기능

- 글 작성
→ Database 생성
→ API 생성
- 랜덤 아이디 생성
→ 화면에서 랜덤 문자열 생성하기
→ Ajax로 요청 시 데이터 서버로 전달하기
- 게시된 글별로 좋아요 적용
→ 게시글 별로 'like'를 적용하기

익명게시판

새 글 작성

글을 작성하세요

작성하기

h4931ie80e

elice 익명게시판

2021-06-13 12:39:31

좋아요 2 개

좋아요

✓ API 생성하기

example

```
@app.route("/", methods=["GET"])
def my_router():
    return render_template("index.html")
# return render_template("index.html", data_list = data)
# return jsonify(data)
```

- **@app.route**로 함수와 url을 연결합니다.
- **methods**에서 **request**방식을 제한해 줍니다.
- return 문으로 응답을 결정합니다. 이때, **render_template**와 **jsonify** 가 많이 사용됩니다.

02

데이터베이스 구성하기



✔ 데이터베이스 구성하기

데이터들을 저장할 공간을 만들어 주어야 합니다.

어떤 데이터를 **어떤 형태**로 저장할지

구체적인 계획을 세워보도록 하겠습니다.

✓ 글 (POST) 모델 만들기

필요한 정보

게시글 ID

작성자 ID

게시글 내용

작성된 시간

게시글 좋아요 수

models.py

```
from flask_sqlalchemy import SQLAlchemy
from datetime import datetime

db = SQLAlchemy()

class Post(db.Model):
    __tablename__ = 'secret_post'
    id = db.Column(db.Integer, primary_key=True,
                   nullable=False, autoincrement=True)
    author = db.Column(db.String(256), nullable=False)
    content = db.Column(db.Text(), nullable=False)
    like = db.Column(db.Integer, default=0)
    created_at = db.Column(db.DateTime,
                           default=datetime.utcnow)
```

03

화면 만들기



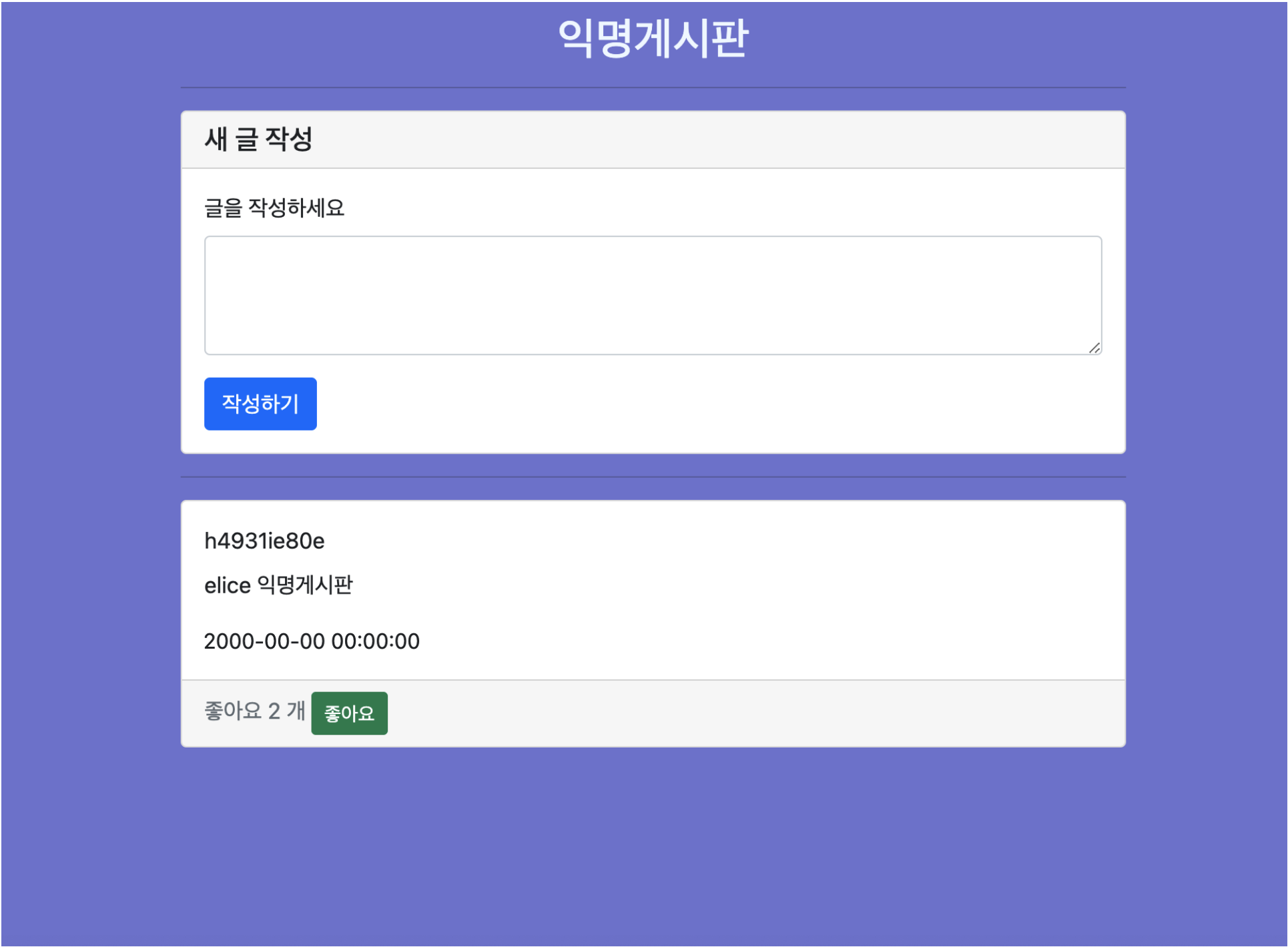
✓ 화면 작성하기

화면은 **HTML, CSS, Javascript로 이루어져 있다**는 것을 기억하시나요?

익명 게시판 화면에 대한 설명을 듣고 같이 만들어 보도록 하겠습니다.

✓ 우리가 만들 화면은?

주제 : 모두가 이용 할 수 있는 익명게시판



배경화면

색상, 정렬

글 작성

글 전송, ID 생성

작성된 글 읽기

글 읽어 오기, 정렬

작성된 글 좋아요 기능

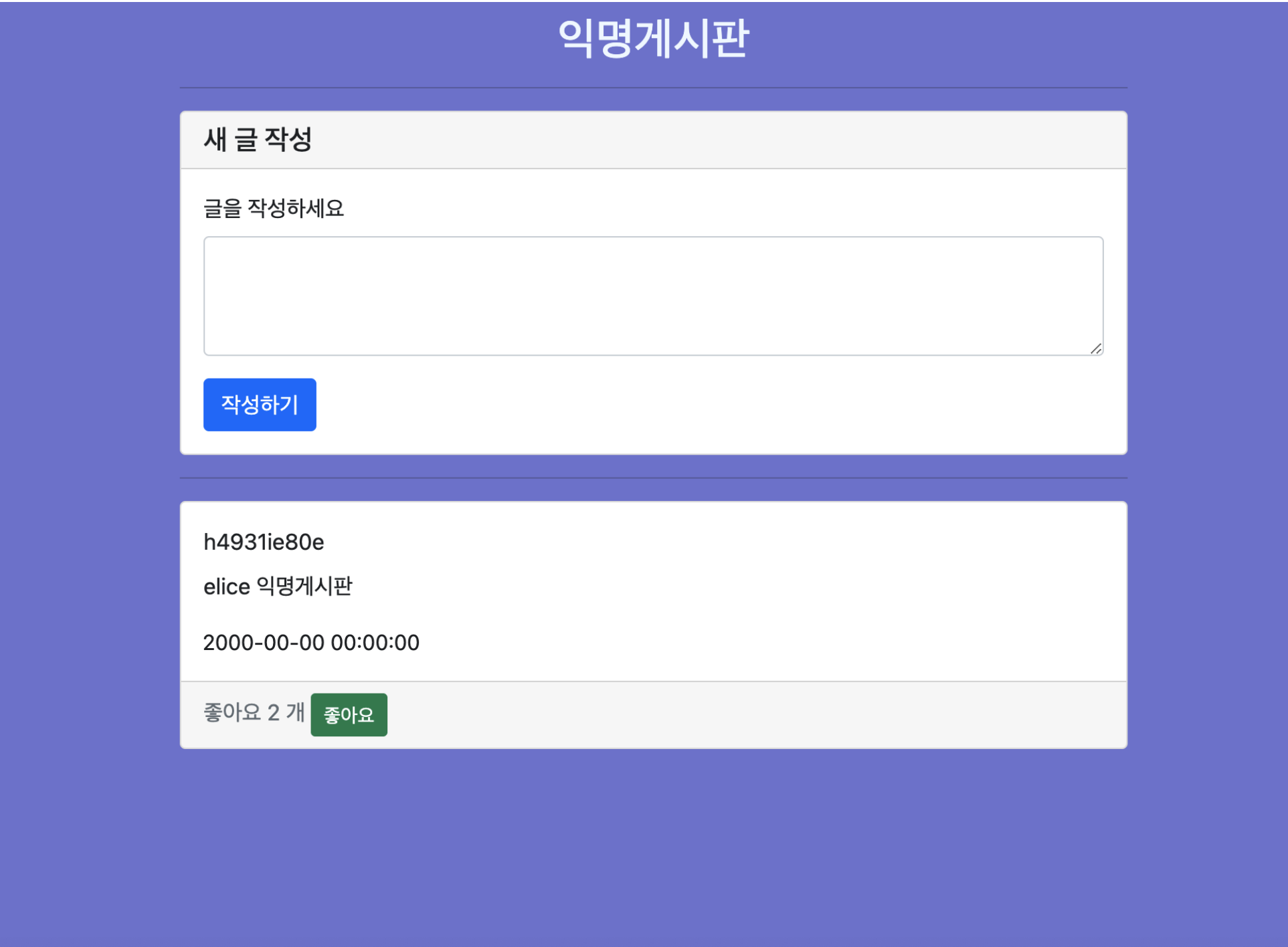
글 좋아요 Update

04

기능 만들기 1 - 글 쓰기



✓ 기능 만들기 1 - 글 쓰기



글쓰기 카드

- jQuery를 사용해서 글 작성 공간 값 가져오기
`$("#content").val()`
- Javascript의 Math.random() 함수를 이용해서 문자 랜덤 생성하기
`Math.random().toString(20).substr(2, 10)`
- Ajax를 사용해서 Flask 서버의 API와 연동하기
→ Flask API 생성하기(API url : /create/post)
→ Flask API와 Ajax 연결하기

✓ 데이터베이스 저장하기 예제

example

```
content = request.form['content']  
post = Post()  
post.content = content  
db.session.add(post)  
db.session.commit()
```

- request로 받아온 데이터를 변수(content)에 저장합니다.
- 저장할 모델을 선언 (Post()) 합니다.
- 모델 안에 데이터 넣어줍니다.
- 세션에 추가 / 저장합니다.

05

기능 만들기 2 - 글 읽기



✓ 기능 만들기 2 - 글 읽기

익명게시판

새 글 작성

글을 작성하세요

작성하기

h4931ie80e

elice 익명게시판

2000-00-00 00:00:00

좋아요 2 개

좋아요

글 읽기 카드

- flask의 jinja2 template 사용하기

```
{% for data in data_list %}
  {{ data }}
{% endfor %}
```
- flask API의 render_template에서
데이터베이스에 있는 값 넘겨주기

```
render_template("index.html",data_list=data)
```

✓ 데이터베이스 읽어 오기 예제

python

```
data = db.session.query(모델이름).filter(조건).all() # 데이터값 읽어 오기 -> list
render_template('html이름', data_list = data)
```

html

```
{{ data_list }}
{% for d in data_list %}
    {{ d }}
{% endfor %}
```

- **query()** 안에 내가 원하는 데이터를 가지고 있는 모델을 작성합니다.
- 가져온 data를 render_template()에 전달합니다.
- html에서 jinja template 문법으로 데이터 읽어옵니다.

06

기능 만들기 3 - 좋아요 누르기



✓ 기능 만들기 3 - 좋아요 누르기

익명게시판

새 글 작성

글을 작성하세요

작성하기

h4931ie80e

elice 익명게시판

2000-00-00 00:00:00

좋아요 2 개

좋아요

글 읽기 카드 - 좋아요

- 좋아요 버튼에 함수 만들어주기
updatePostLike(id) → 해당 게시글의 id
- Database Update 기능 만들기
db.session.commit()

✓ 데이터베이스 업데이트 예제

example

```
data = db.session.query(모델 이름).filter(조건).first()
data.like = data.like+1 # 데이터값 수정
db.session.commit()
```

- **query()** 안에 내가 원하는 데이터를 가지고 있는 모델을 작성합니다.
- **filter()** 안에 읽어올 데이터의 조건들을 작성합니다.
- data를 불러왔다면, 가져온 데이터에 있는 값을 수정합니다.
- db.session에 이미 데이터를 불러왔기 때문에 add() 없이 **commit()** 합니다.

크레딧

/* elice */

코스 매니저

이재성

콘텐츠 제작자

이바울, 이재성

강사

이바울

감수자

이바울

디자이너

강혜정

연락처

TEL

070-4633-2015

WEB

<https://elice.io>

E-MAIL

contact@elice.io

